

Notes on Neural Networks: A light introduction to Recurrent Neural Networks

Alessio Micheli

micheli@di.unipi.it

2025



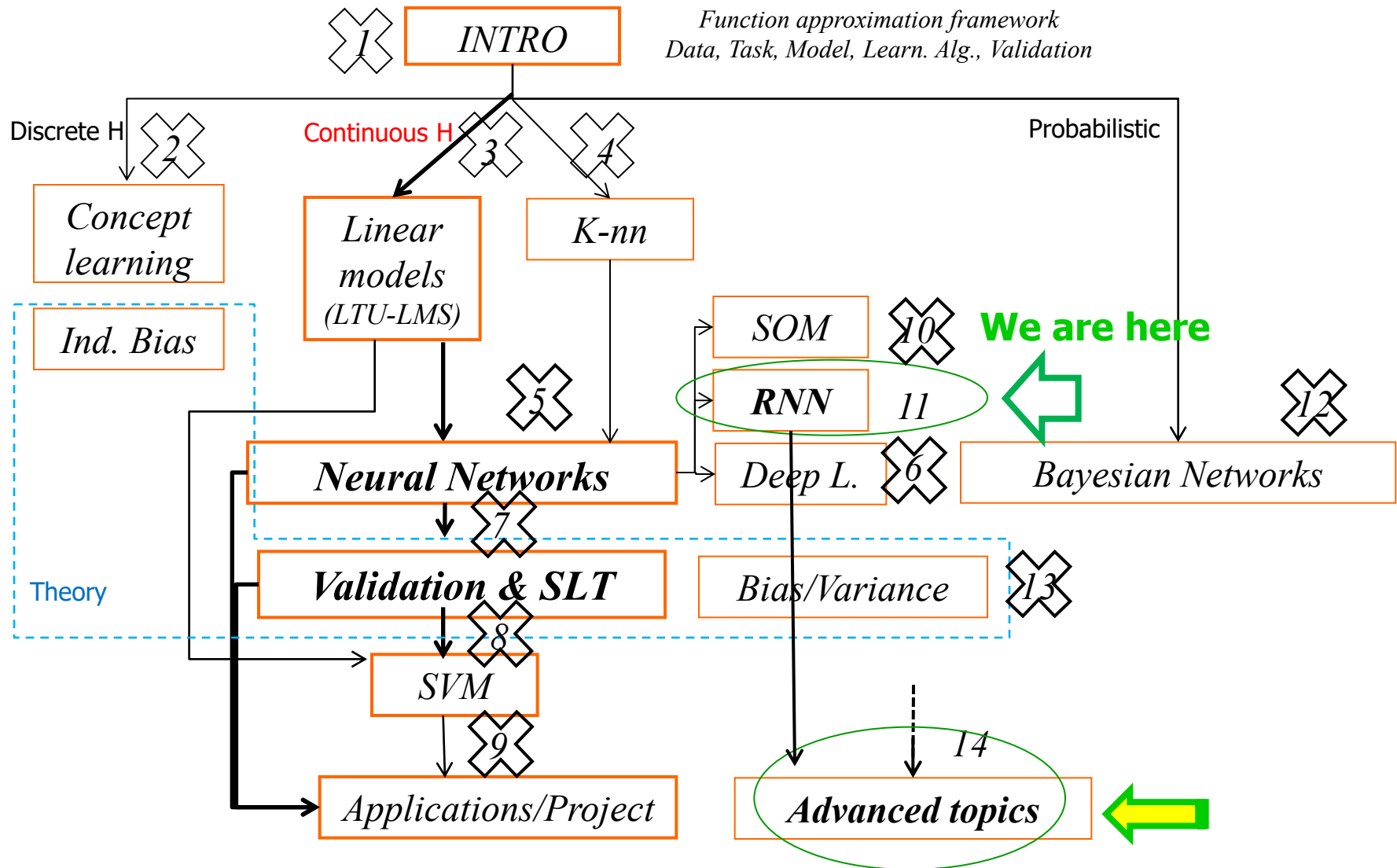
Dipartimento di Informatica
Università di Pisa - Italy

ML Course structure

Where we are and go



Dip. Informatica
University of Pisa

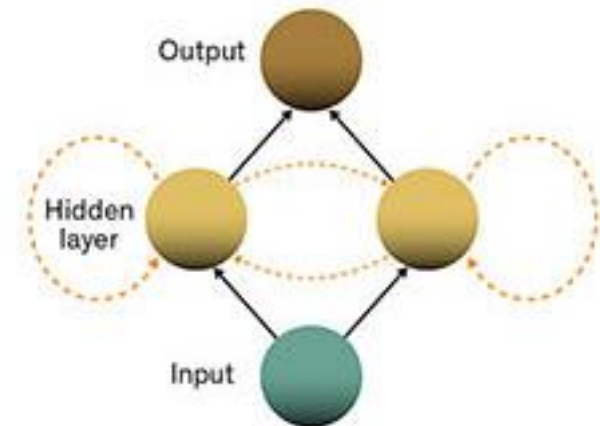


Intro to RNN



- IEEE Spectrum (magazine) 26 Jan 2016
- **“The Neural Network That Remembers”**
 - **With short-term memory, recurrent neural networks gain some amazing abilities**

A recurrent neural network includes connections between neurons in the hidden layer [yellow arrows], some of which **feed back** on themselves.

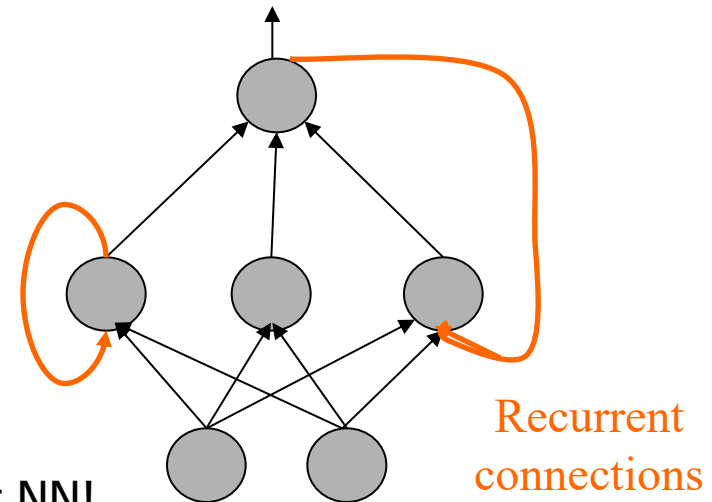


Feedforward versus Recurrent

- **Feedforward**: direction: input $\rightarrow$ output
- **Recurrent** neural networks: A different category of architecture, based on the addition of *feedback loops* connections in the network topology,
 - The presence of **self-loop** connections provides the network with dynamical properties, letting a memory (**states**) of the past computations in the model.
 - This allows us to extend the representation capability of the model to the processing of sequences (and structured data).

Recurrent neural networks:

- See (these) separate slides in this course
- They will be the subject of the (further developed) ISPR/**CNS** courses
- **Neurobiological** plausibility
 - nervous system/biological NN are recurrent NN!



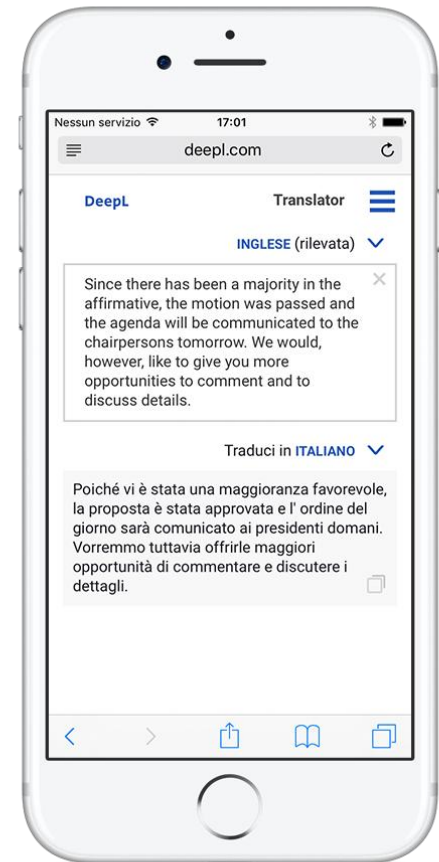
Why sequential data?

- Whenever the output of the model depends on the *history* of the inputs – e.g. *time*: dynamical models
- Or the domain is composed by varying-size sequences
 - Dynamical processes. Signal processing (Filters, Control). Robotics*
 - Language* (Speech recognition, NLP, Formal languages, IR*)
 - Vision, Reasoning (temporal events in AI):
 - Temporal series: financial forecasting, Signal processing *
 - Genomics/Proteomics (Bioinformatics*)
- 2 examples:
 - (human) Activity recognition (from sensor data)
 - Structure protein prediction from the sequence of aminoacids

Applicative Examples

RNN (and related approaches) have been the reference approach for sequence processing,
especially for (state-of-the-art results):

- Speech recognition/processing
- Text processing
 - Machine translation etc....
- Or even more (fun):
 - music composition
 - text/speech generation
 - ...

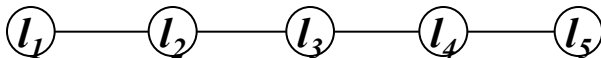


From flat to structured data

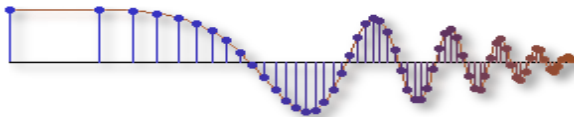
- **Flat: vectors** (as in the rest of this ML course)
- **Structured:** Sequences, trees, graphs, multi-relational data

stringa_in_italiano

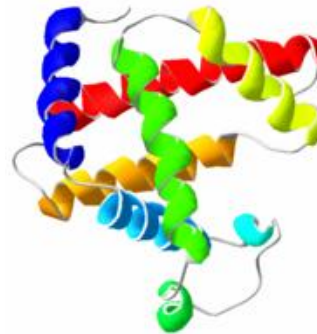
Strings



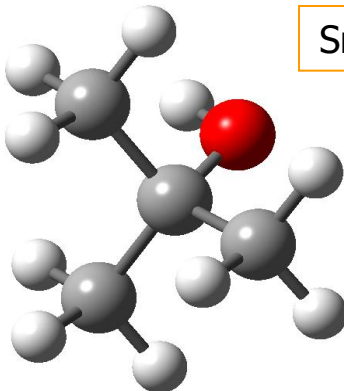
Series/
temporal stream



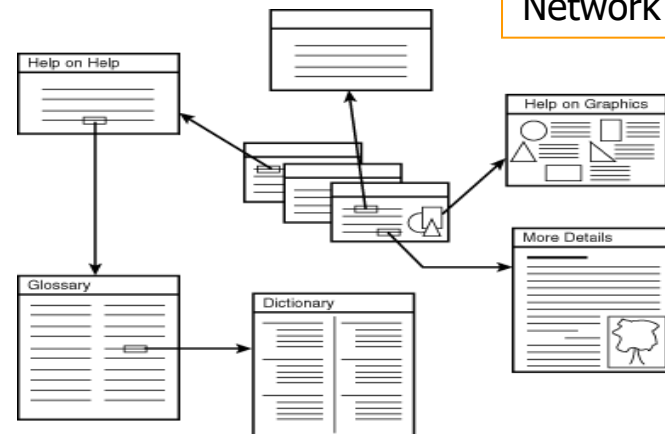
Proteins



Small molecules



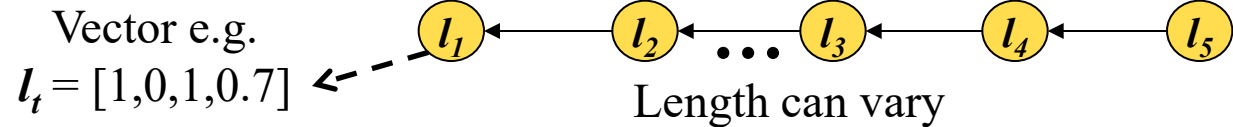
Network data



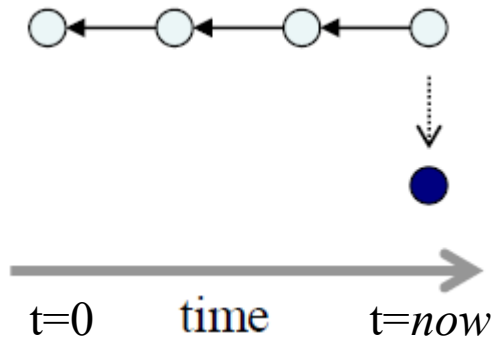
A new input Domain & Trasductions

- Data: discrete sequence of vectors (serial order, e.g. time)

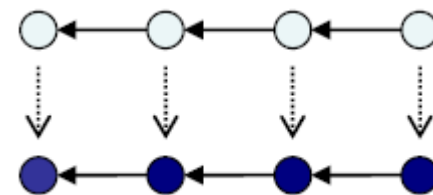
- Tasks:



Sequences recognition/classification:
output at the end



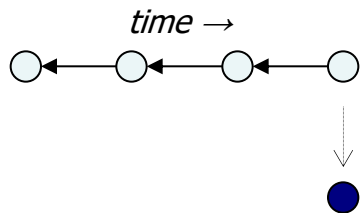
Sequence Transduction, Next Step Prediction:
output for each input step



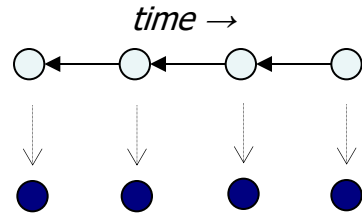
Each output is a vector (discrete or continuous values)

Transduction in the sequential domain: a mapping
Input sequence $\rightarrow$ Output value or sequence

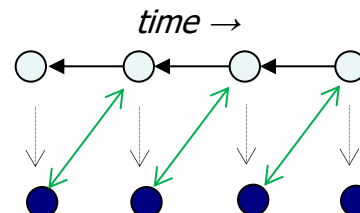
Sequence Transduction: details



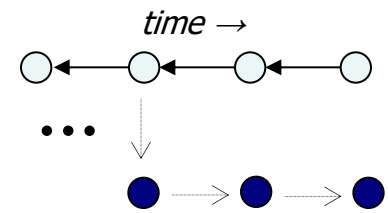
Sequence
classification



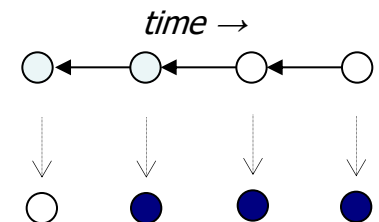
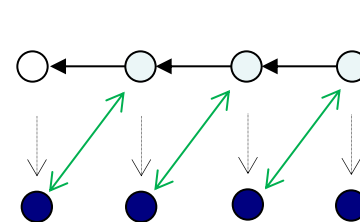
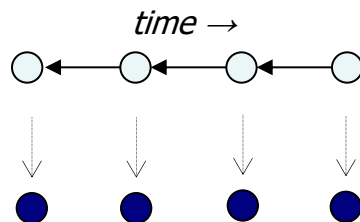
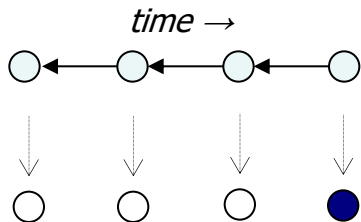
IO-transduction
(Input-Output isomorphic)



Next-step
Prediction (*)



Sequence
generation



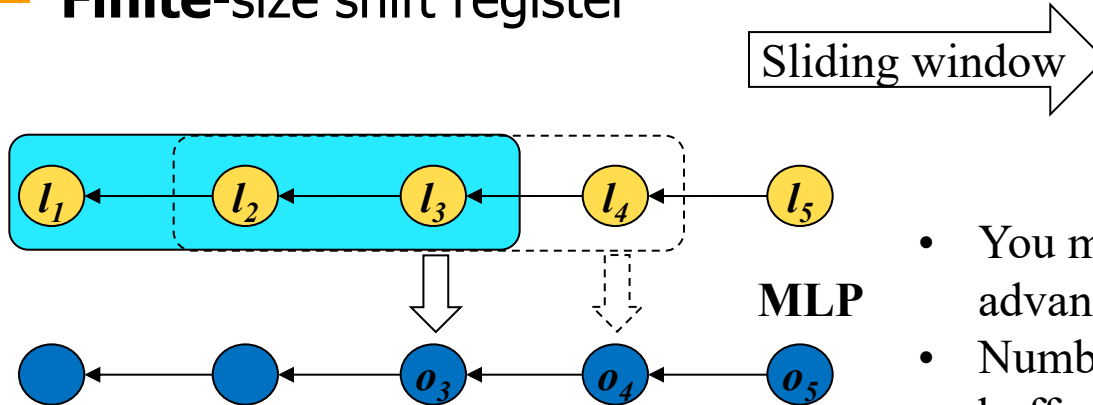
A **general transduction** where empty/full node
can represent the different cases above

**Autoregressive*: next-step prediction with just 1 sequence →
nowadays popular task with generative models!

Memory in NN: 1. IDNN

- Aim: Output depends on the previous inputs
- Temporal processing in NN
- 1. Finite memory/spatial approach: **See beginning of CNN lectures**
 - Input delay neural networks (in Time-delay NN = TDNN class)
 - Since delays are in the NN connections to act as input memory

– Finite-size shift register



**CNN extended this idea
to 2D images!**

- You must know the window dim in advance
- Number of weights depends on the buffer size!!!
- If the buffer includes sufficient information we can solve with a MLP training algorithm! (e.g. NETalk, 1988)

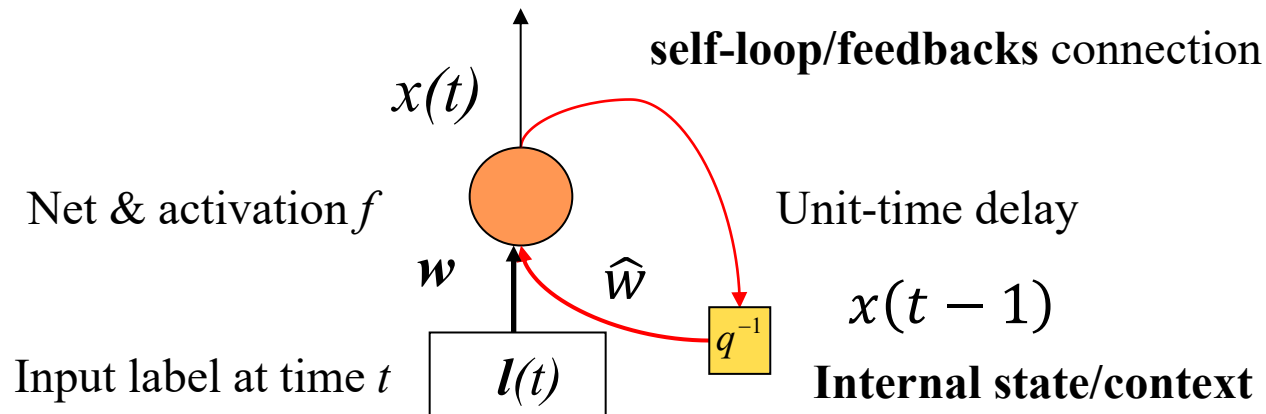
Memory in NN: 2. RNN

- 1 recurrent unit



Dip. Informatica
University of Pisa

- A Recurrent Neural **unit** with feedbacks in the architecture
- It uses current input + states information



$$x(0)=0$$

$$x(t) = \tau(l(t), x(t-1)) = f(\mathbf{w}^T l(t) + \hat{w}x(t-1) + \theta)$$

New part !!

f = (sigmoidal)
activation function
 $\hat{w}$ = recurrent weight
 θ = bias

- The **state** summarizes the past information (context): it changes its value according to the stream of inputs
- The *encoding* of the past is **adaptive** (due to the free weights of the model)

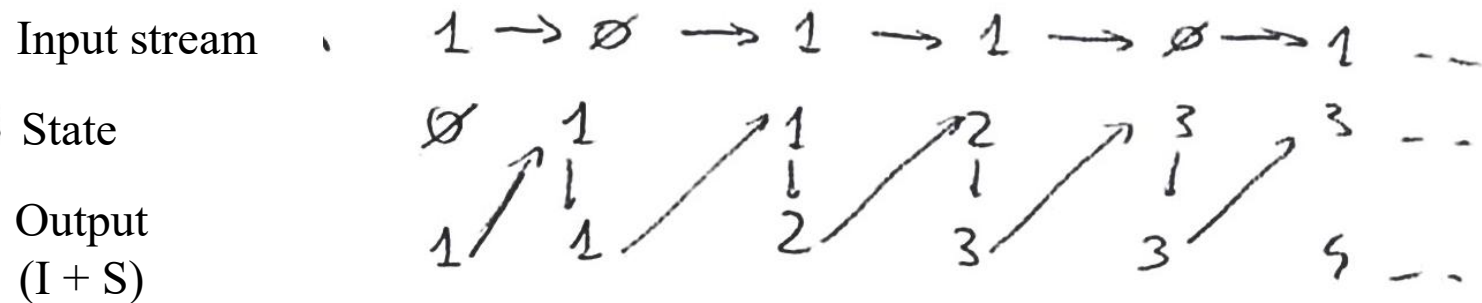
An example

Exercise: realize a 1-unit RNN (find the values for the 2 weights w e $\hat{w}$) which outputs the sum of "1" received in input so far.

Input stream: 1 and 0

Drawn the flow of input/state/output values time-by-time for an example (101101)

Solution ;-)

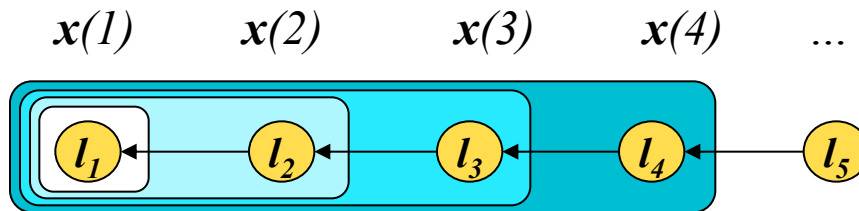


- Note that the state accumulates the sum of "1" received in input so far:
- The RNN summarizes the past subsequence information
- IDNN cannot do it for varying size sequences!!!

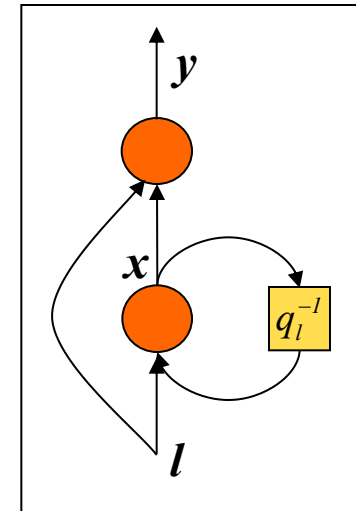
State transition system - recurrent model

Given $\mathbf{x}(0) = \mathbf{0}$

$$\begin{cases} \mathbf{x}(t) = \tau(\mathbf{x}(t-1), \mathbf{l}(t)) \\ \mathbf{y}(t) = g(\mathbf{x}(t), \mathbf{l}(t)) \end{cases}$$



State formation over time
Sub-sequences encoding/ embedding



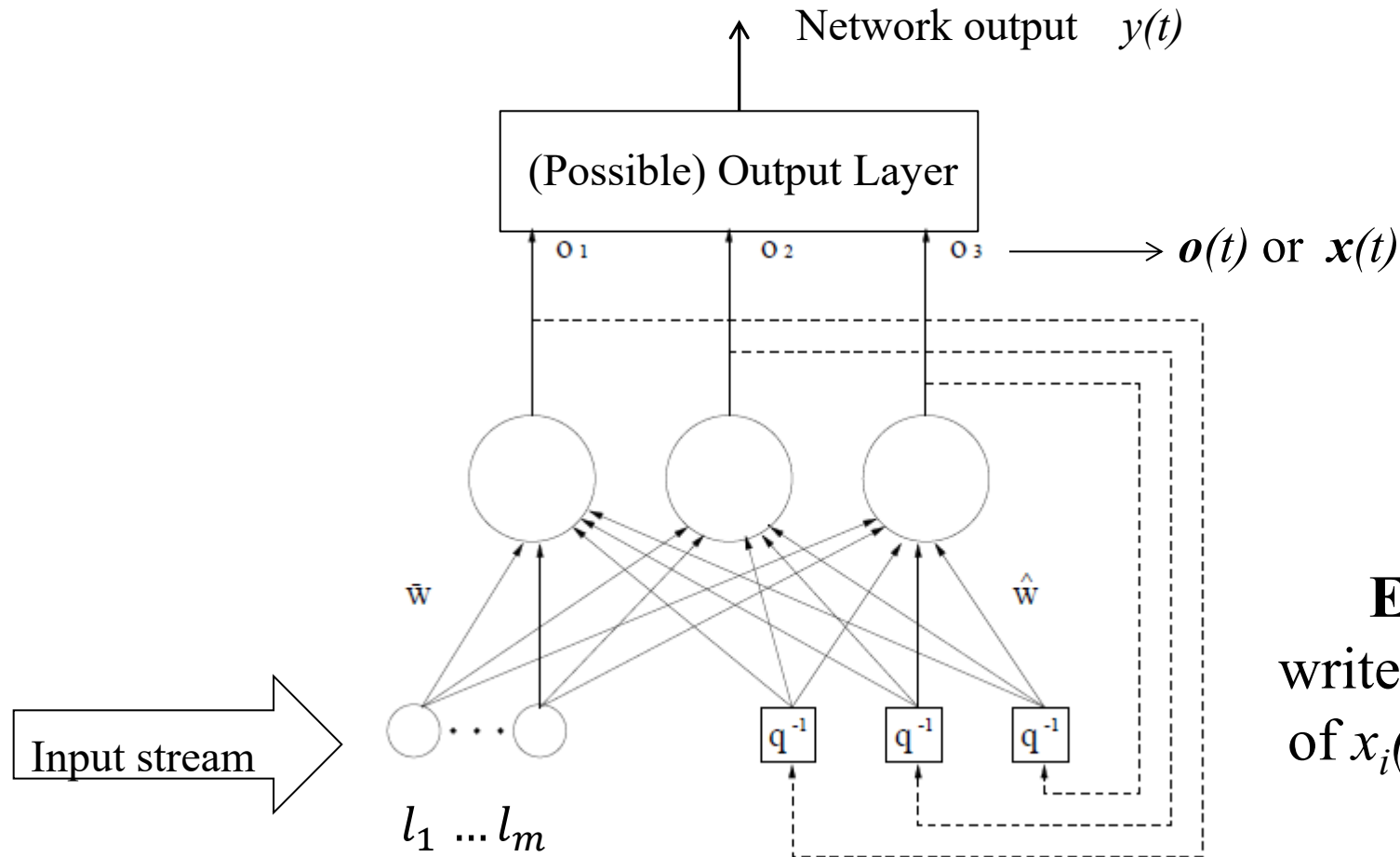
Graphical
model

E.g.
also **HMM**

- Tau (τ) is the *state transition* (next-state) function realized by a NN
- The state summarizes the past inputs
- Note that the state have continuous values (real numbers)
- $\mathbf{x}(t)$ can be a set of states (to compose a network of recurrent units)

Recurrent Neural Network

- Many hidden recurrent neurons ("**Simple RNN**" – **Elman**)



Exercise:
write the equation
of $x_i(t)$ (and then
of $x(t)$)

Properties

- Of course many architectures are possible, however the “Simple RNN” seen before is already powerful:
- **Universal approximators** of *non-linear* dynamical systems
- **Turing equivalence** (e.g. any automata can be simulated)
- Further: RNN as a (non-autonomous) *dynamical non-linear system* → dynamics studies → chaos theory (e.g. fractals theory ...)

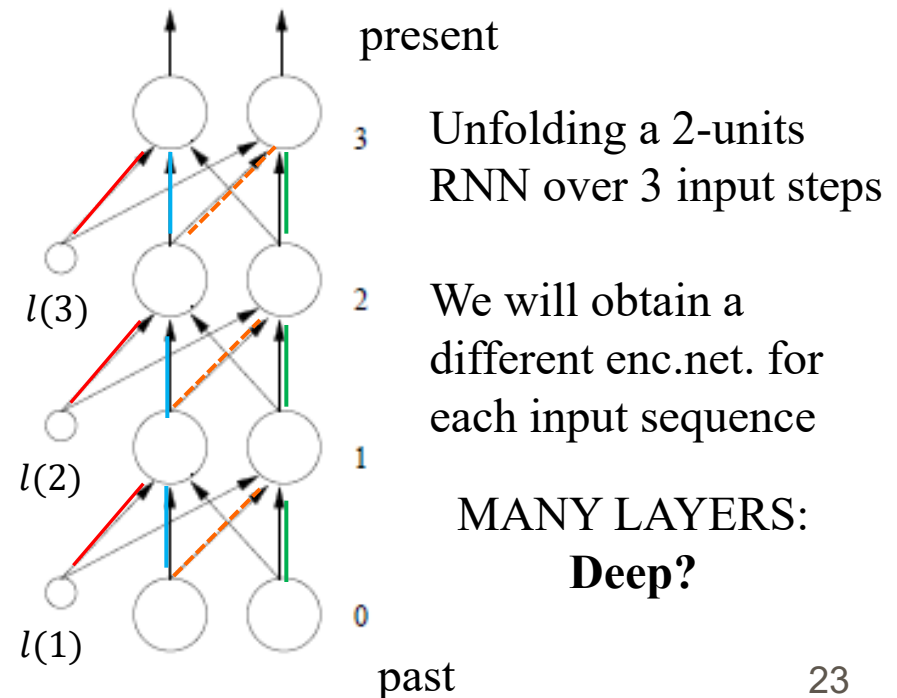
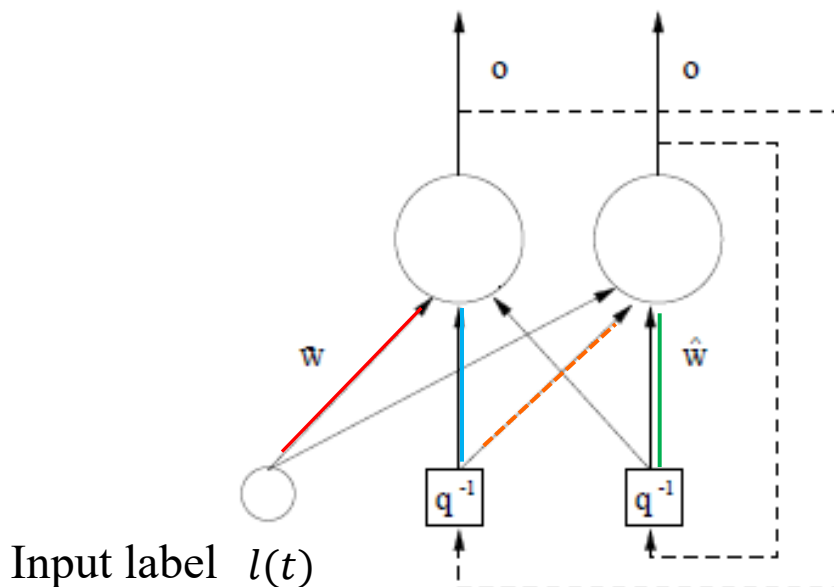
Properties of RNN

Recurrent NN models are based on the following **assumptions**:

- **Causality**: A system is causal if the output at time t_0 (or node v) only depends on inputs at time $t < t_0$ (depends only on v and its descendants)
 - necessary and sufficient for *internal state*
- **Stationarity**: time invariance (after model training), i.e. the state transition function τ is independent on node v of the sequence
 - i.e. τ is the same in any time t , regardless of the dimension of sequences (useful to process data of variable length with a fixed size of the model)
- **Adaptivity**: transition functions are realized by NN (with free weight parameters), hence they are **learnt** from data
 -

Unfolding

- Unrolling through time (i.e. input sequence) of the same model
- Build a feedforward MLP (called *encoding network*) equivalent (on the given k steps) to the RNN (given a specific input sequence)
- 1 model-replica for each input step
- Stationarity $\rightarrow$ **weight sharing** (across unfolded layers) !!
- Nice! : On this enc.net. we can apply back-prop learning algorithm !!!



Learning algorithms

The learning algorithm must account the set of encode transitions developed by the model for each step of the inputs.

- **BPTT** (back-propagation through time) and
- **RTRL** (real time recurrent learning)

supervised learning algorithms are designed for recurrent neural networks.

They compute, in different style, (the same) gradient values of the output errors across an **unfolded** over time network that is equivalent to the recurrent one (encoding networks).

- The vanish of gradient over this deep network can make difficult to learn Long-term dependencies

Advanced models

Rapid growing research field, many variants:

- **LSTM** (long-short term memory): try to solve the gradient vanish by using “gate units” able to select the past/gradient flow.
 - Note: born to address the need to take long memory: not suitable for short series!
 - Hence, despite their popularity, do not use them acritically
 - See applications later (next slide)
- **GRU** (gated recurrent units): simplified LSTM
- **BRNN**: Bi-directional RNN: to consider both the left and right context
- ... Often fusing DeepNN + RNN
- But also (to deal with gradient vanish issues):
 - Hessian-Free optimizer
 - Pre-training techniques ...
- Many included in the popular DL ML libraries: Caffe(2), PyTorch, Keras, Matlab, CNTK, ...

Popular applications

It has been for many years a reference method for sequence processing, especially for (with often state-of-the-art results)

- Speech recognition/processing
- Text processing (Machine translation etc....)
- Some data (even if not updated):
 - In 2015 LSTM(RNN) used in 2 billion Android phones. 1 billion iPhones (siri).
 - In 2016 almost 30% of the awesome Google datacenters computational power for inference was used for LSTM(RNN).
 - 2017: LSTM(RNN) used in Alexa, Facebook (30 billion translations every week).
- And lots of fun...E.g.
 - Generating text/speech/music ...
 - See also our previous examples on the caption generations, language learning (deepNN lectures),

Another nice example

recurrent neural network handwriting generation demo

Human user inputs a text and
the algorithm generates a handwritten version of it.

Generate pen trajectories by *predicting one* (x,y)
point at a time

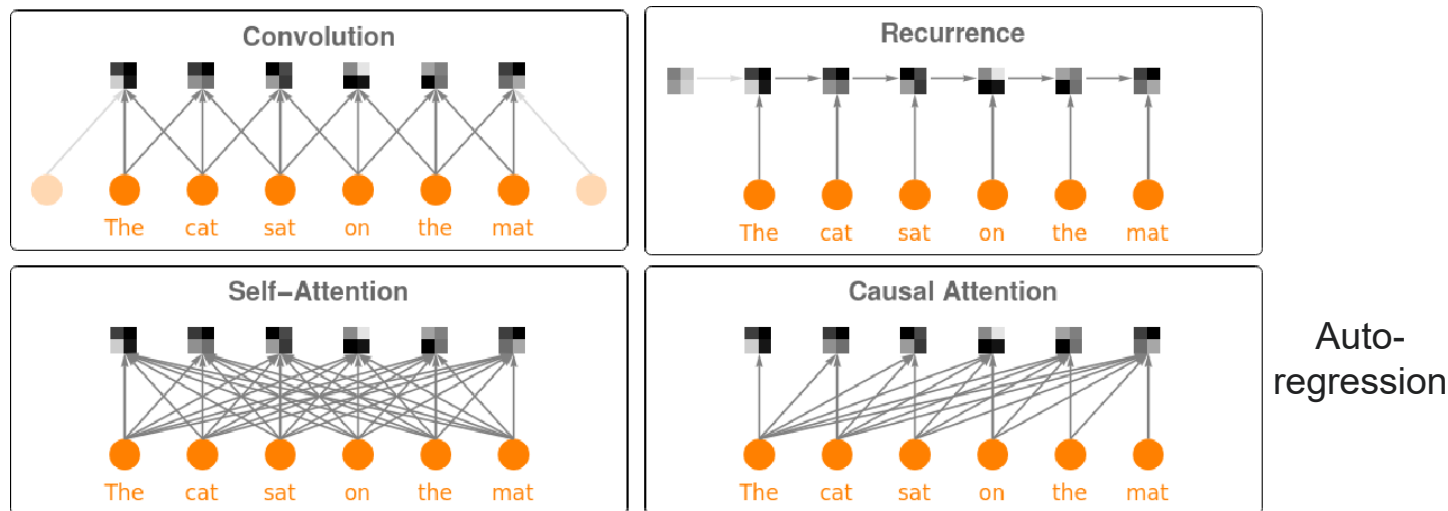
Graves, *Generating Sequences With Recurrent Neural Networks*
<https://arxiv.org/abs/1308.0850>

Related approaches

- **SOM/unsupervised NN** for sequential domains
- Hidden Markov Models- **HMM** (model the probability distribution of state transitions)
- **Transformers**
- **Randomized (R)NN: Reservoir Computing**: ESN/LSM (see next slides)
- Distance based models (string matching)
- **Kernel** for strings
- Grammatical inference
- **ILP**

Transformers

- A **transformer** is a **deep learning** architecture, proposed in 2017, that relies on the parallel multi-head **attention** mechanism
- Attention mechanism allows a model to access any preceding point along the sequence



- *Encoder and/or decoder* + feedforward neural networks complete the architecture for different purposes

Transformers: Attention

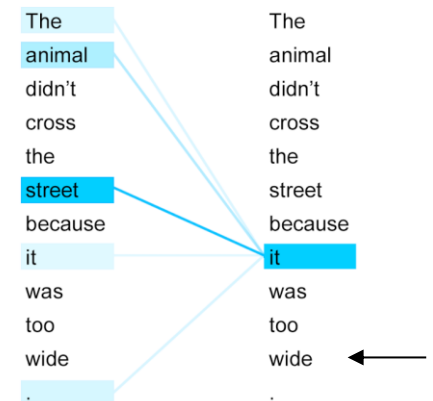
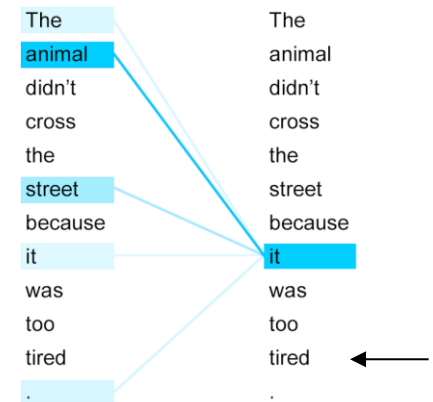
- The **attention layer** weights all previous states according to a learned measure of relevance, providing, potentially, *information about far-away tokens*.

This proved to be especially useful in **language translation**, where far-away context can be essential for the meaning of a word in a sentence

- First used to improve RNN models and then even alone (Paper: *Attention is all you need*)
- Processing all tokens simultaneously, calculating weights between them in successive layers, or better between their embedding
 - just scaled dot-product units using query and word vectors to search *the highest correlations amongst words* within a sentence. **E.g** "see that girl" whereas "that" is identified as the correlated to word "girl", or the examples on the right: *why the difference?*
- It can be applied also to patches (instead of words) of **images** (vision transformers), etc.

See further courses in these applicative fields e.g.

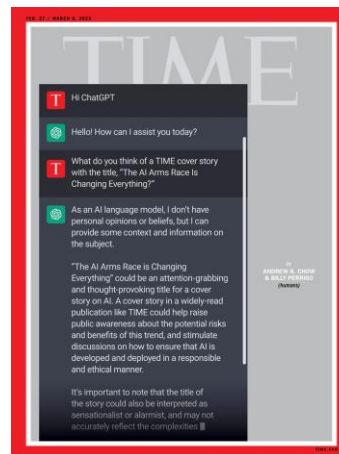
ISPR/GDL & HLT !



GPT

Generative Pre-trained Transformer

Used to generate text, speech, images, code ...



A 2023 *Time* cover:
"The AI Arms Race Is
Changing Everything"

Randomized RNN Toward ESN



Dip. Informatica
University of Pisa

- Can we exploit directly this *state machine* to encode sequences and then use the resulting embedding to learn the output mapping?
- Yes, even with *random connected recurrent networks* with fixed (untrained) **random weights!!!** (please refer to NN lectures for “*random weights NN*”)
- Fractal machine + **Reservoir Computing**(RC) /ESN/liquid machine class of RNN
- Very **efficient** (no hidden recurrent unit learning at all), with the capability to solve well task *under specific conditions (bias)*.
 - Ability to intrinsically discriminate among different input sequences in a suffix-based fashion without adaptation of the recurrent parameters (see later)

Echo State Networks

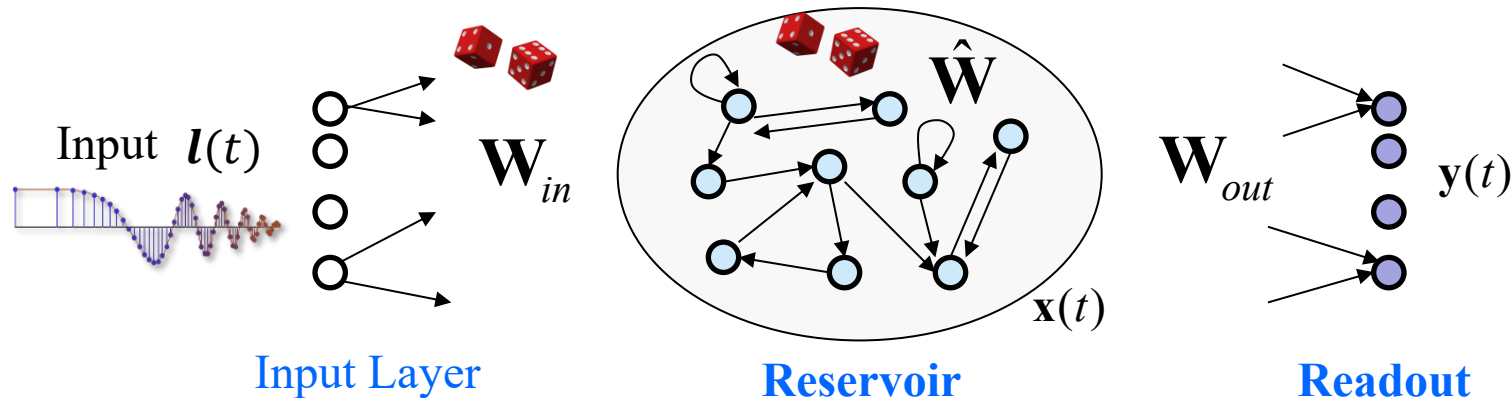
- Emerging paradigm for efficiently modeling Recurrent NNs
- An **Echo State Network (ESN)** consists in a

Large *reservoir* of sparsely connected recurrent units

Untrained after random* initialization

Simple feed-forward *readout* of linear units

Trained by efficient linear methods, e.g. *ridge regression (closed-form solution)*



- Echo State Property:** contractivity of the state transition functions (limiting *spectral radius* ρ of reservoir weights, i.e. stability of the dynamical system)
- Recurrent state asymptotically depends (*echo*) *only* on the *history of the inputs* (and not on the initial state conditions)

Reservoir Computing

Liquid-based reservoirs

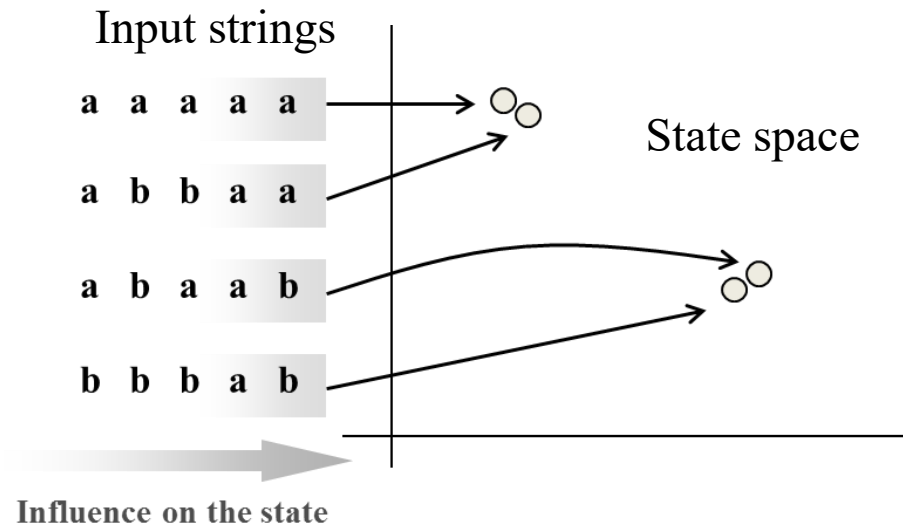
The liquid surface shows different spatiotemporal patterns (reservoir states) according to how it is perturbed and their temporal orderings make the patterns of the liquid surface different.



C. Fernando and S. Sojakka, Lecture Notes in Computer Science 2801 (Springer, Berlin, 2003), p. 588.

Credits. G. Milano

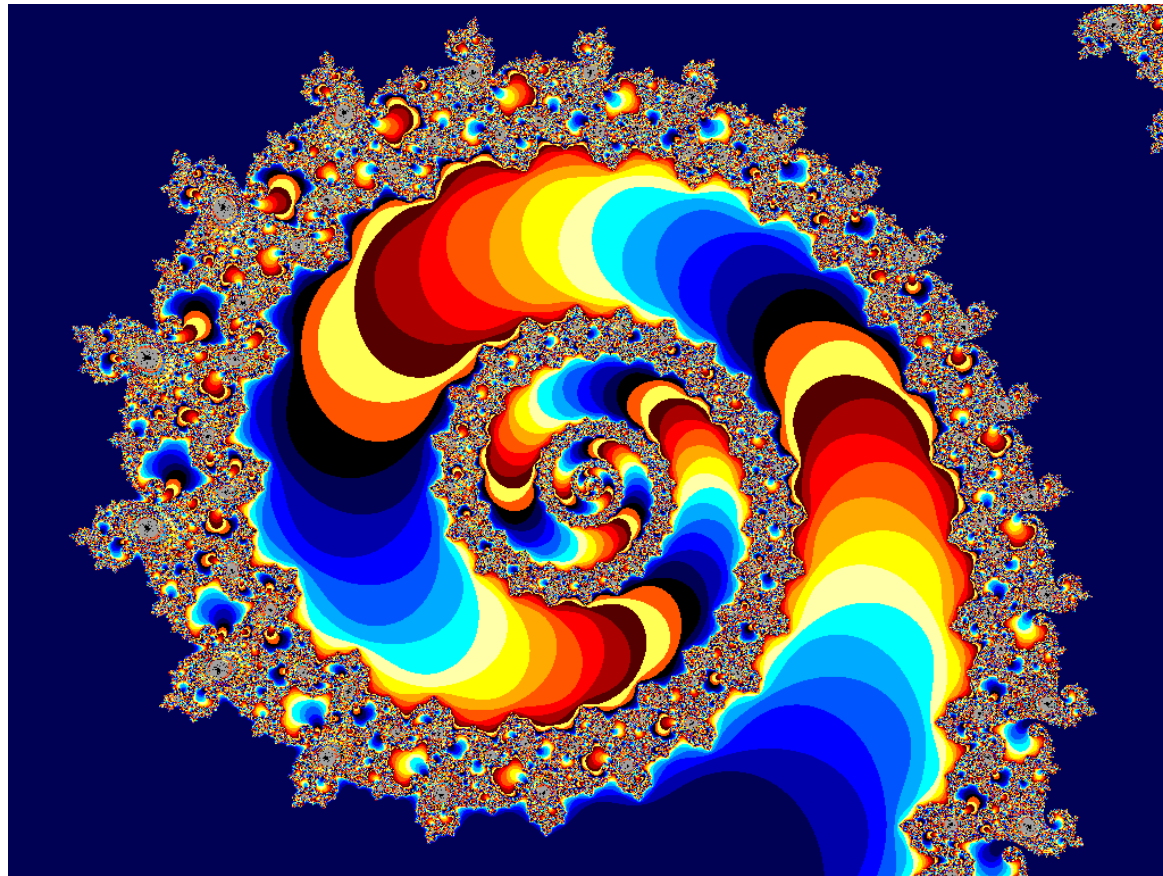
Insight: the state space Markovian organization



- Reservoir acts as a projection (embedding) of input sequences in the vector of states $\mathbf{x}(t)$ values
- Ability to intrinsically discriminate among different input sequences in a *suffix-based fashion* without training of the recurrent weights
 - The states are close to each other proportionally to the length of the common suffix
 - Easy to learn Markovian task (also for RNN but ESN are of course much more efficient!)
 - Architectural bias for recurrent models with learning !

C. Gallicchio, A. Micheli, "Architectural and Markovian factors of echo state networks", Neural Networks, 2011

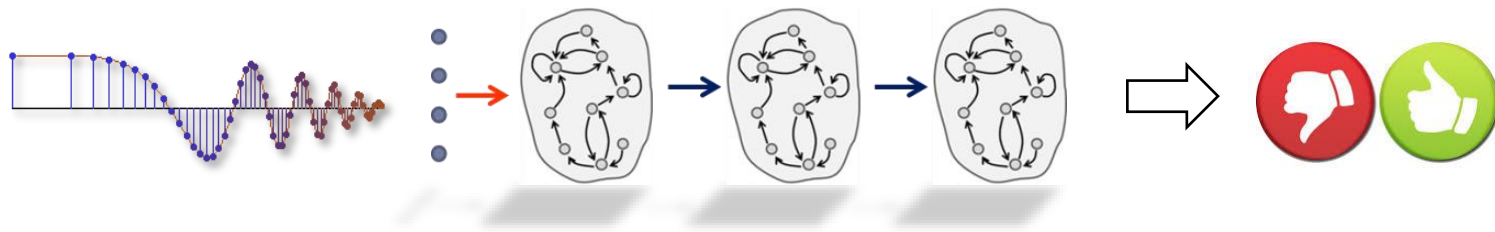
Fractals



ESN/RNN are related to dynamical systems studies, chaos theory, fractals ...

RC + Deep?

- RC+ Deep: Yes, we can.
 - And we have deepRNN power+ efficiency of ESN.
- Analysis by **key questions**, e.g.:
 - Why **stacking** recurrent layers?
 - What is the inherent (independent from learning) architectural **effect of layering**?
- Deep Echo state Networks have been studied (and still continue), with many results introduced by the **CIML group** in Pisa:
- Deep layered organization of RC models show advantages in terms of
 - occurrence of multiple time-scale
 - increasing of richness of the dynamics (hierarchical)



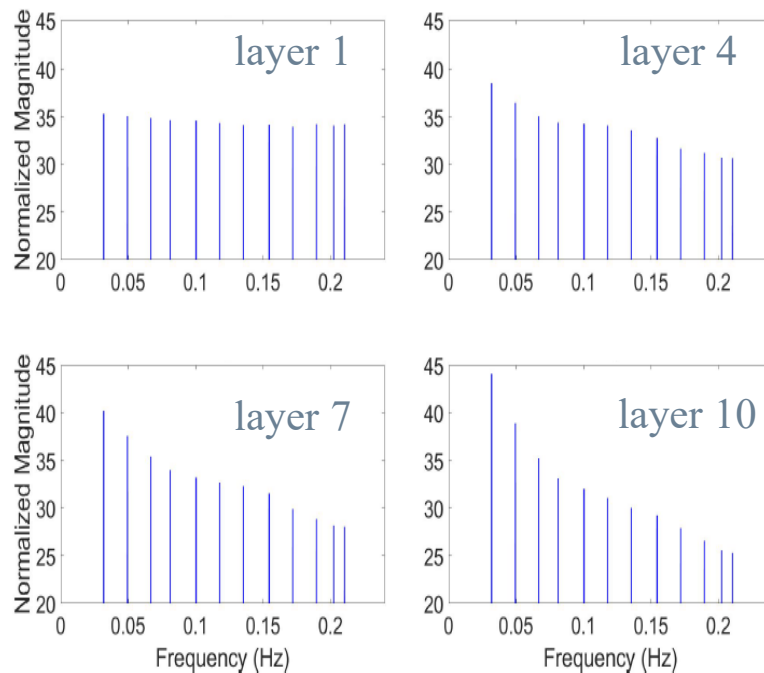
Hierarchical Temporal Features in DeepESN



Dip. Informatica
University of Pisa

Structured representation of temporal data through the deep architecture (multi time-scale)

Frequency Analysis (here in the linear DeepESN [*])

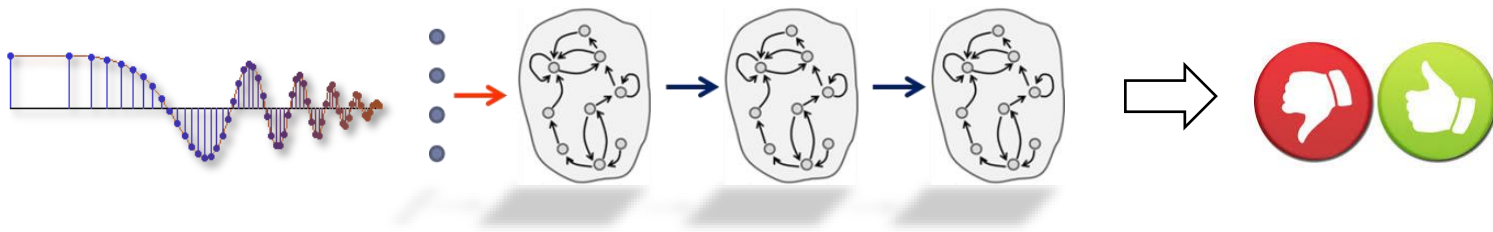


- Diversified magnitudes of FFT components in states (from inputs)
- **Multiple frequency representation**
- Ordered along the network's hierarchy
- **Higher layers focus on lower frequencies (larger time scale)**

[*] Gallicchio, Micheli, Pedrelli. **Hierarchical Temporal Representation in Linear Reservoir Computing**
WIRN (2017) <https://arxiv.org/abs/1705.05782>

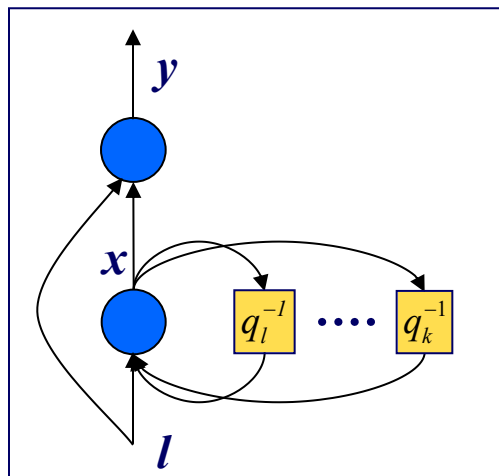
Deep RC references

- Many results:
 - C. Gallicchio, A. Micheli, L. Pedrelli, "*Deep Reservoir Computing: A Critical Experimental Analysis*", Neurocomputing, 2017
 - C. Gallicchio, A. Micheli, "*Echo State Property of Deep Reservoir Computing Networks*", Cognitive Computation, 2017
 - C. Gallicchio, A. Micheli, L. Silvestri, "*Local Lyapunov Exponents of Deep RNN*", 2018.
 - C. Gallicchio, A. Micheli, L. Pedrelli.
Design of deep echo state networks. Neural Networks (2018), Vol. 108, December 2018
 - ... many others
- **A short guide/overview:**
 - C. Gallicchio, A. Micheli, "*Deep Echo State Network (DeepESN): A Brief Survey*", [arXiv:1712.04323](https://arxiv.org/abs/1712.04323)

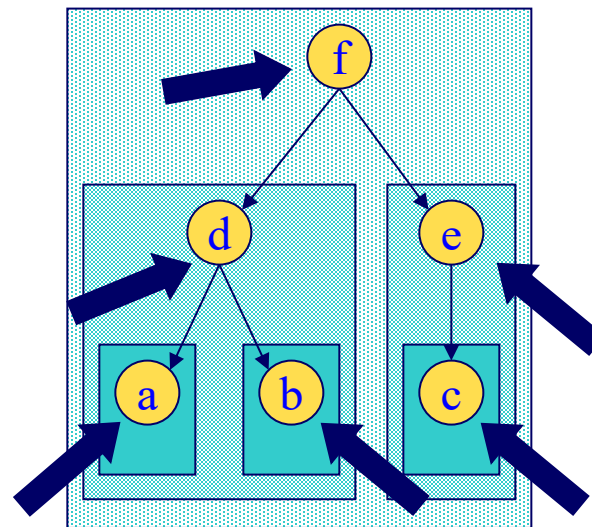


Toward Structured Domains (SD)

- Can we extend this recurrent approach to rooted trees?
- Yes, *Recursive Neural Networks*
 - Boot-up encoding of input trees
 - Extending states for children of vertices of the tree



Graphical model for trees

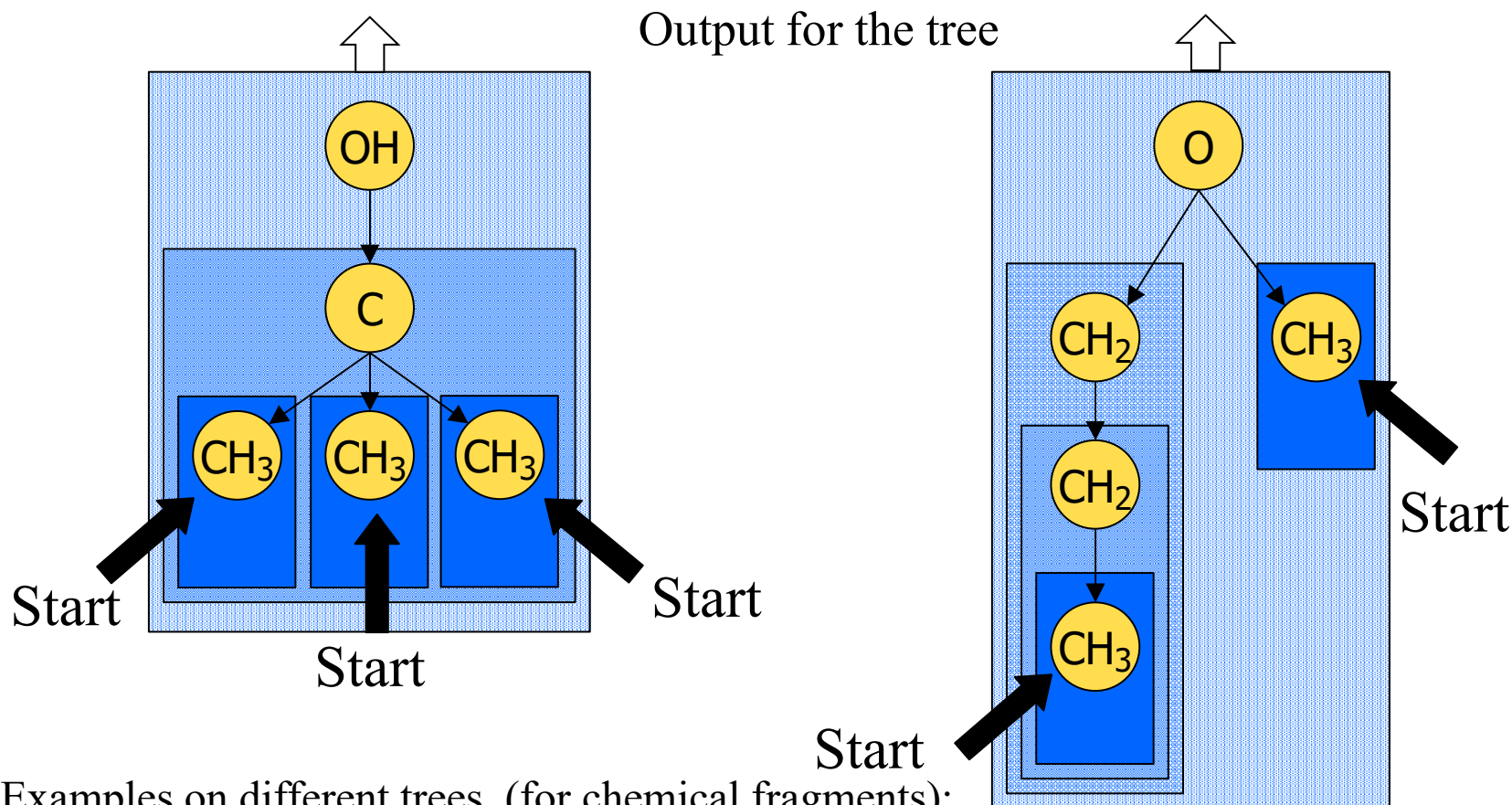


Unfolding the encoding process through structure

If we change the
tree structure,
the encoding
change
accordingly !
See next slide



RecNN over different structures



Examples on different trees (for chemical fragments):

Unfolding (encoding) through different structures.

Weight sharing: units are the same for all the vertices of a tree and for all the trees in the data set!!!

Bibliography (RNN)

- **Haykin:**

- Chap 1.5 & 1.6 (2nd and 3th edit)
- Chap. 15: Dynamically Driven Recurrent Network (both 2nd and 3th edition). **15.1 and 15.2**

Other references:

- DL book: Chapter 10 (*not needed for this class*).
- [**historical paper**] Elman, J.L. (1990). "Finding Structure in Time". *Cognitive Science* vol. **14**: 179–211 (pdf on-line through search engines)
- Unfolding: historical: pages 354 → 361 of the Parallel Distributed Processing, Vol.1, Mit Press, 1987
- - [**In book**, copy upon request] Introduction: Chapt. 2 of the book Kolen, Kremer: A Field Guide to Dynamical Recurrent Networks. IEEE press 2001

Bibliography (Reservoir Computing/ESN)



Dip. Informatica
University of Pisa

- H. Jaeger, H. Haas, *Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication*, Science, vol. 304, no. 5667, pp. 78–80, 2004 [short introduction to the model with an applicative example. Support material on-line therein]
- M. Lukosevicius, H. Jaeger, 2009. *Reservoir computing approaches to recurrent neural network training*. Computer Science Review 3 (3), 127 –149. [Survey on the Reservoir Computing (at 2009)]
- C. Gallicchio, A. Micheli, "Architectural and Markovian factors of echo state networks", Neural Networks, Volume 24, Issue 5, Pages 440-456, 2011 [Empirical study and Markovian characterization of the ESN]
- M. Lukosevicius, 2012. *A Practical Guide to Applying Echo State Networks*. <http://minds.jacobs-university.de/uploads/papers/PracticalESN.pdf>
- DeepRC: see before ("Deep RC references slide")

Bibliography (...)

- Many topics can be subject of study and development
- **Bibliography on demand:** micheli@di.unipi.it
- **MUCH MORE on RNN** in the Unipi courses:
 - ISPR (Intelligent Systems for Pattern Recognition) → GDL
 - CNS (Computational Neuroscience)
- **Thesis** on RC
and related arguments are available
(models for sequence/structured data)



*“tie a knot in your
handkerchief”*

RNN Software, Hardware

The main models (Elman recurrent NN - SRN, LSTM, GRU, ...) are included in the popular Deep Learning / Machine Learning libraries as:

- Caffe(2), PyTorch, **Keras**, Pybrain, **Matlab**, Theano, CNTK, ...
- Our software for RC: see CIML page
<http://www.di.unipi.it/groups/ciml>
- Hardware implementations: via *Photonic* Reservoir Computing:
 - speed, latency, energy consumption (power efficiency) advantages.
(See at CNS course)
- ...

More information

Alessio Micheli
micheli@di.unipi.it



Dipartimento di Informatica
Università di Pisa - Italy



**Computational Intelligence &
Machine Learning Group**